assignment3_part2

October 23, 2026

1 Assignment 3: Dealing with Vector and Matrix Real-World Data (Part II)

1.1 Similarity of vectors

Now that we have represented the user-restaurant ratings as a matrix, where each row vector represents the ratings of one restaurant, we can compute the similarity of two restaurants by calcluating the similarity of the two corresponding vectors. As there are various measurements of similarity, let's try a few of them.

First, let's import the packages and dependencies that will be used in this part of the assignment.

[1]:
```
from math import sqrt
```

NOTE: These are all the imports we need to make for this assignment (Part II). You should not make other imports in your submitted notebook. You will receive 0 points for the exercises if your solution includes additional imports.

1.1.1 Dot Product

Dot product, also known as inner product, is defined as

$$\vec{X} \cdot \vec{Y} = \sum_i x_i y_i$$

and it can be calculated with the following `dot` function

[2]:
```
def dot_prod(vec_x, vec_y):
    """
    For this assignment, you can assume the validity of the input, that is,
    `vec_x` and `vec_y` are both arrays of float values with the same length.
    """
    prod = 0
    for i in range(len(vec_x)):
        prod += vec_x[i] * vec_y[i]
    return prod
```

This function is to show you how dot product can be calculated. In fact, this function has already been implemented by the `numpy` package, and we can easily use `np.dot(vec_x, vec_y)` to calculate the dot product, which is much more efficient.

1.1.2 Exercise 2. Implementing similarity/distance metrics (45 pts, 15 pts for each)

Now that we have shown you how to implement dot product, can you implement the other similarity/distance metrics introduced in the lectures?

Note that *distance* and *similarity* are the opposite sides of the same concept. For each exercise, BE CAREFUL to see whether we are asking you to implement a **distance** or **similarity** metric.

Exercise 2(a) Manhattan Distance Recall from the lecture, Manhattan Distance is defined as follows:

$$d(\vec{X}, \vec{Y}) = \sum_i |x_i - y_i|$$

[5]:
```
def manhattan_distance(vec_x, vec_y):
    """
    For this assignment, you can assume the validity of the input, that is,
    `vec_x` and `vec_y` are both arrays of float values with the same length.
    """
    sum_abs = 0
    for i in range(len(vec_x)):
        # YOUR CODE HERE
        #raise NotImplementedError()
        sum_abs += abs(vec_x[i] - vec_y[i])
    return sum_abs
```

[6]:
```
# This code block tests whether the `manhattan_distance` function is␣
 ↪implemented correctly.
# We hide some tests, so passing all the displayed assertions does not␣
 ↪guarantee full points.

assert abs(manhattan_distance([1, 1, 1, 1], [1, 1, 1, 2]) - 1) < 1e-8
assert abs(manhattan_distance([1, 0, 1], [2, 0, 2]) - 2) < 1e-8
```

Exercise 2(b) Euclidean Distance Recall from the lecture, Euclidean Distance is defined as

$$d(\vec{X}, \vec{Y}) = \sqrt{\sum_i (x_i - y_i)^2}$$

[7]:
```
def euclidean_distance(vec_x, vec_y):
    """
    For this assignment, you can assume the validity of the input, that is,
    `vec_x` and `vec_y` are both arrays of float values with the same length.
    """
    sum_square = 0

    for i in range(len(vec_x)):
        # YOUR CODE HERE
        sum_square += (vec_x[i] - vec_y[i])**2
```

```
    return sqrt(sum_square)
```

[8]:
```
euclidean_distance([1, 0, 1], [2, 0, 2])
```

[8]:
```
1.4142135623730951
```

[9]:
```
# This code block tests whether the `manhattan_distance` function is␣
 ↪implemented correctly.
# We hide some tests, so passing all the displayed assertions does not␣
 ↪guarantee full points.

assert abs(euclidean_distance([1, 1, 1, 1], [1, 1, 1, 2]) - 1) < 1e-8
assert abs(euclidean_distance([1, 0, 1], [2, 0, 2]) - sqrt(2)) < 1e-8
```

Exercise 2(c) Cosine Similarity And finally, the cosine similarity is defined as:

$$\cos(\vec{X}, \vec{Y}) = \frac{\sum_i x_i y_i}{\sqrt{\sum_i x_i^2} \cdot \sqrt{\sum_i y_i^2}}$$

[10]:
```
def cosine_similarity(vec_x, vec_y):
    """
    For this assignment, you can assume the validity of the input, that is,
    `vec_x` and `vec_y` are both arrays of float values with the same length.
    You can further assume that both vectors are not zero vector.
    """
    sum_x2 = 0
    sum_y2 = 0
    sum_xy = 0
    cos_sim = -1

    for i in range(len(vec_x)):
        sum_x2 += vec_x[i]**2
        sum_y2 += vec_y[i]**2
        sum_xy += vec_x[i]*vec_y[i]

    cos_sim = sum_xy / (sqrt(sum_x2)*sqrt(sum_y2))
    # YOUR CODE HERE
    #raise NotImplementedError()
    return cos_sim
```

[11]:
```
cosine_similarity([1, 0, 1], [2, 0, 2])
```

[11]:
```
0.9999999999999998
```

[12]:
```
# This code block tests whether the `manhattan_distance` function is␣
↪implemented correctly.
# We hide some tests, so passing all the displayed assertions does not␣
↪guarantee full points.

assert abs(cosine_similarity([1, 1, 1, 1], [1, 1, 1, 2]) - 0.944911182523068) <␣
↪1e-8
assert abs(cosine_similarity([1, 0, 1], [2, 0, 2]) - 1) < 1e-8
```

1.1.3 Final Note

In this assignment, we have implemented several similarity/distance metrics by hand. In reality, however, it's better to use more efficient and professionally maintained implementations to calculate the metrics. Many packages (such as NumPy, SciPy, and scikit-learn) have done this for you. With many lower-level optimization tricks, they are much more efficient. In fact, you are encouraged to check out a few of them and compare with your implemented functions. Just make sure not to use them directly in your submission of Exercise 2.

- The SciPy documentation on distance metrics can be found here.
- NumPy does not implement these metrics directly, but it is very handy (and efficient) to implement with NumPy's vector operators. For example, cosine similarity can be implemented as `np.dot(x, y)/(np.linalg.norm(x) * np.linalg.norm(y))`, euclidean distance as `np.norm(x - y)`.
- Scikit-learn offers many utility functions under `sklearn.metrics.pairwise` You can read more about it here. These APIs are very handy to calculate the *pairwise* similarity within a list (or between two lists) of vectors.

When you use an API, **be careful to check whether it returns a distance or similarity metric.**